

机器学习练习- 朴素贝叶斯

代码更新地址: <https://github.com/fengdu78/WZU-machine-learning-course>

代码修改并注释: 黄海广, haiguang2000@wzu.edu.cn

1. 朴素贝叶斯法是典型的生成学习方法。生成方法由训练数据学习联合概率分布 $P(X, Y)$, 然后求得后验概率分布 $P(Y|X)$ 。具体来说, 利用训练数据学习 $P(X|Y)$ 和 $P(Y)$ 的估计, 得到联合概率分布:

$$P(X, Y) = P(Y)P(X|Y)$$

概率估计方法可以是极大似然估计或贝叶斯估计。

2. 朴素贝叶斯法的基本假设是条件独立性,

$$\begin{aligned} P(X = x|Y = c_k) &= P(X^{(1)} = x^{(1)}, \dots, X^{(n)} = x^{(n)}|Y = c_k) \\ &= \prod_{j=1}^n P(X^{(j)} = x^{(j)}|Y = c_k) \end{aligned}$$

这是一个较强的假设。由于这一假设, 模型包含的条件概率的数量大为减少, 朴素贝叶斯法的学习与预测大为简化。因而朴素贝叶斯法高效, 且易于实现。其缺点是分类的性能不一定很高。

3. 朴素贝叶斯法利用贝叶斯定理与学到的联合概率模型进行分类预测。

$$P(Y|X) = \frac{P(X, Y)}{P(X)} = \frac{P(Y)P(X|Y)}{\sum_Y P(Y)P(X|Y)}$$

将输入 x 分到后验概率最大的类 y 。

$$y = \arg \max_{c_k} P(Y = c_k) \prod_{j=1}^n P(X_j = x^{(j)}|Y = c_k)$$

后验概率最大等价于0-1损失函数时的期望风险最小化。

模型:

- 高斯模型
- 多项式模型
- 伯努利模型

```
In [52]: import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from collections import Counter
import math
```

```
In [53]: # data
def create_data():
    iris = load_iris()
    df = pd.DataFrame(iris.data, columns=iris.feature_names)
    df['label'] = iris.target
    df.columns = [
        'sepal length', 'sepal width', 'petal length', 'petal width', 'label'
    ]
    data = np.array(df.iloc[:, :])
    #print(data)
    return data[:, :-1], data[:, -1]
```

```
In [54]: X, y = create_data()

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

```
In [55]: X_test[0], y_test[0]
```

```
Out[55]: (array([5.4, 3. , 4.5, 1.5]), 1.0)
```

参考: <https://machinelearningmastery.com/naive-bayes-classifier-scratch-python/>

GaussianNB 高斯朴素贝叶斯

特征的可能性被假设为高斯

概率密度函数:

$$P(x_i|y_k) = \frac{1}{\sqrt{2\pi\sigma_{yk}^2}} \exp\left(-\frac{(x_i - \mu_{yk})^2}{2\sigma_{yk}^2}\right)$$

数学期望(mean): μ

方差: $\sigma^2 = \frac{\sum(X-\mu)^2}{N}$

```
In [56]: class NaiveBayes:
    def __init__(self):
        self.model = None

    # 数学期望
    @staticmethod
    def mean(X):
        return sum(X) / float(len(X))

    # 标准差 (方差)
    def stdev(self, X):
        avg = self.mean(X)
        return math.sqrt(sum([pow(x - avg, 2) for x in X]) / float(len(X)))

    # 概率密度函数
    def gaussian_probability(self, x, mean, stdev):
        exponent = math.exp(-(math.pow(x - mean, 2) /
                                     (2 * math.pow(stdev, 2))))
        return (1 / (math.sqrt(2 * math.pi) * stdev)) * exponent
```

```

# 处理X_train
def summarize(self, train_data):
    summaries = [(self.mean(i), self.stdev(i)) for i in zip(*train_data)]
    return summaries

# 分类别求出数学期望和标准差
def fit(self, X, y):
    labels = list(set(y))
    data = {label: [] for label in labels}
    for f, label in zip(X, y):
        data[label].append(f)
    self.model = {
        label: self.summarize(value)
        for label, value in data.items()
    }
    return 'gaussianNB train done!'

# 计算概率
def calculate_probabilities(self, input_data):
    # summaries:{0.0: [(5.0, 0.37),(3.42, 0.40)], 1.0: [(5.8, 0.449),(2.7, 0.449)]}
    # input_data:[1.1, 2.2]
    probabilities = {}
    for label, value in self.model.items():
        probabilities[label] = 1
        for i in range(len(value)):
            mean, stdev = value[i]
            probabilities[label] *= self.gaussian_probability(
                input_data[i], mean, stdev)
    return probabilities

# 类别
def predict(self, X_test):
    # {0.0: 2.9680340789325763e-27, 1.0: 3.5749783019849535e-26}
    label = sorted(self.calculate_probabilities(X_test).items(),
                    key=lambda x: x[-1])[-1][0]
    return label

def score(self, X_test, y_test):
    right = 0
    for X, y in zip(X_test, y_test):
        label = self.predict(X)
        if label == y:
            right += 1
    return right / float(len(X_test))

```

```
In [57]: model = NaiveBayes()
```

```
In [58]: model.fit(X_train, y_train)
```

```
Out[58]: 'gaussianNB train done!'
```

```
In [59]: print(model.predict([4.4, 3.2, 1.3, 0.2]))
```

```
0.0
```

```
In [ ]:
```

```
In [60]: model.score(X_test, y_test)
```

```
Out[60]: 0.9777777777777777
```

scikit-learn实例

```
In [61]: from sklearn.naive_bayes import GaussianNB, BernoulliNB, MultinomialNB  
# 高斯模型、伯努利模型和多项式模型
```

```
In [62]: clf = GaussianNB()  
clf.fit(X_train, y_train)
```

```
Out[62]: ▼ GaussianNB  
GaussianNB()
```

```
In [63]: clf.score(X_test, y_test)
```

```
Out[63]: 0.9777777777777777
```

```
In [64]: clf.predict([[4.4, 3.2, 1.3, 0.2]])
```

```
Out[64]: array([0.])
```

参考

[1] Andrew Ng. Machine Learning[EB/OL].
StanfordUniversity,2014.<https://www.coursera.org/course/ml>

[2] 李航. 统计学习方法[M]. 北京: 清华大学出版社,2019.